

# Python 模拟合约交易核心逻辑

<https://gemini.google.com/app/ea9a4a5028542db9>

*User prompt:* 讲到这里, 我认为我们下一步, 是需要去用Python先让一两个合约市场跑起来, 我们暂时不要考虑在区块链上部署合约, 只需要去做实验即可。

*Response:* 非常赞同你的思路! 在开发复杂的 Web3 或量化交易系统时, “先在本地用纯代码模拟核心逻辑, 再去考虑区块链部署”是最接地气、也最容易出成果的办法。这样做能让我们把精力 100% 集中在业务逻辑 (比如: 保证金怎么扣、爆仓线怎么算、多空怎么撮合) 上, 而不用去跟漫长的区块链交易确认和高昂的 Gas 费死磕。

为了让这一两个合约市场真正“跑起来”进行实验, 我们通常可以用 Python 的 面向对象编程 (OOP) 来抽象出三个核心角色:

1. 行情源 (**Market Data Engine**): 模拟价格的变动。
2. 交易市场 (**Exchange/Market**): 维护订单簿、计算持仓、计算保证金和处理爆仓。
3. 交易账户 (**Trader/User**): 模拟用户下单、查看资产。

我们可以设计一个简单的实验脚本。这里我为你梳理了一个高内聚、易扩展的 Python 最小可行性实验框架。

## 核心实验代码实现

你可以把下面这段代码直接复制到本地的 Python 环境 (比如 Jupyter Notebook 或 .py 文件) 中运行。它模拟了一个永续合约市场的核心逻辑: 价格变动 -> 产生浮动盈亏 -> 触发保证金不足爆仓。

```
import time
import random

class Position:
    """持仓类: 记录单个用户的持仓情况"""
    def __init__(self, symbol):
        self.symbol = symbol
        self.size = 0.0          # 持仓大小 (正数为多头, 负数为空头)
        self.entry_price = 0.0   # 平均开仓价
        self.margin = 0.0        # 该持仓分配的保证金

    @property
    def direction(self):
        if self.size > 0: return "LONG"
        if self.size < 0: return "SHORT"
        return "NONE"
```

```

class PerpetualMarket:
    """永续合约市场类：处理业务逻辑"""
    def __init__(self, symbol, leverage=10, maintenance_margin_rate=0.05):
        self.symbol = symbol
        self.leverage = leverage # 初始杠杆倍数
        self.mmr = maintenance_margin_rate # 维持保证金率（低于这个比例就会被爆仓）
        self.current_price = 100.0
        self.positions = {} # user_id -> Position

    def set_price(self, new_price):
        """模拟市场价格变动"""
        self.current_price = new_price
        self.check_liquidations()

    def open_position(self, user_id, side, size, price):
        """开仓逻辑"""
        if user_id not in self.positions:
            self.positions[user_id] = Position(self.symbol)

        pos = self.positions[user_id]

        # 简单起见，这里假设是首次开仓或同向加仓
        actual_size = size if side == "LONG" else -size

        # 计算需要的初始保证金 = (大小 * 价格) / 杠杆
        required_margin = (size * price) / self.leverage

        # 更新持仓均价和大小
        if pos.size == 0:
            pos.entry_price = price
            pos.size = actual_size
        else:
            # 均价计算公式
            total_cost = (abs(pos.size) * pos.entry_price) + (size * price)
            pos.size += actual_size
            pos.entry_price = total_cost / abs(pos.size)

        pos.margin += required_margin
        print(f"【开仓成功】用户 {user_id} | {side} | 数量: {size} | 均价: {pos.entry_price} | 投入保证金: {required_margin:.2f}")

    def get_pnl_and_equity(self, user_id):
        """计算用户的未实现盈亏 (uPnL) 和 仓位权益"""
        pos = self.positions.get(user_id)
        if not pos or pos.size == 0:
            return 0.0, 0.0

        # 盈亏计算
        if pos.direction == "LONG":
            unrealized_pnl = pos.size * (self.current_price - pos.entry_price)
        else: # SHORT
            unrealized_pnl = abs(pos.size) * (pos.entry_price -

```

```

self.current_price)

    # 仓位权益 = 初始保证金 + 未实现盈亏
    equity = pos.margin + unrealized_pnl
    return unrealized_pnl, equity

def check_liquidations(self):
    """爆仓检查机制"""
    for user_id, pos in list(self.positions.items()):
        if pos.size == 0:
            continue

        u_pnl, equity = self.get_pnl_and_equity(user_id)
        # 维持保证金要求 = 持仓名义价值 * MMR
        notional_value = abs(pos.size) * self.current_price
        maintenance_margin = notional_value * self.mmr

        # 如果当前权益低于维持保证金, 触发强平
        if equity < maintenance_margin:
            print(f"\n💣💣【触发爆仓】用户 {user_id} 被强制平仓!")
            print(f"当前价格: {self.current_price} | 开仓价: {pos.entry_price}")
            print(f"未实现盈亏: {u_pnl:.2f} | 剩余权益: {equity:.2f} (低于维持保证金 {maintenance_margin:.2f})")
            # 清空持仓
            pos.size = 0
            pos.margin = 0

# --- 实验运行模拟 ---
if __name__ == "__main__":
    print("🚀 启动永续合约本地模拟市场...")
    # 创建一个 BTC-USDT 市场, 默认 10 倍杠杆, 维持保证金率 5%
    btc_market = PerpetualMarket(symbol="BTC-USDT", leverage=10,
    maintenance_margin_rate=0.05)

    # 1. 模拟 Alice 在 100 元时看多, 开仓 2 个单位
    print(f"\n--- 初始市场价格: {btc_market.current_price} USDT ---")
    btc_market.open_position(user_id="Alice", side="LONG", size=2, price=100.0)

    # 2. 模拟行情上涨
    print(f"\n--- 模拟行情上涨 ---")
    btc_market.set_price(110.0)
    pnl, equity = btc_market.get_pnl_and_equity("Alice")
    print(f"当前价格: 110.0 | Alice 浮盈: {pnl:.2f} | 仓位总权益: {equity:.2f}")

    # 3. 模拟行情暴跌, 测试爆仓线
    print(f"\n--- 模拟行情暴跌 ---")
    # 10倍杠杆, 多头在跌幅接近 10% 时, 除去 5% 维持保证金, 跌大约 9.5% 就会爆仓 (即价格跌到 90.5 左右)
    for simulated_price in [95.0, 92.0, 90.0]:
        if btc_market.positions["Alice"].size == 0:
            break

```

```
print(f"价格波动至: {simulated_price}...")
btc_market.set_price(simulated_price)
```

## 为什么这个实验对下一步至关重要？

通过这个简单的 Python 脚本，我们其实已经把衍生品最核心的数学模型跑通了。在接下来的实验中，我们可以基于这个框架轻松地叠加更复杂的模块，而不需要动任何智能合约的代码：

- 多市场并行:通过 `eth_market = PerpetualMarket("ETH-USDT")` 就能瞬间再开一个市场。
- 资金费率 (**Funding Rate**) 实验:可以加上定时器，每模拟过去一段时间，就让多头付给空头(或相反)一笔资金费。
- 模拟自动交易员 (**Bot**):写一个简单的循环，让几个 AI 账号基于随机游走(Random Walk)或均线策略在这个市场里自动下单，测试市场的压力承受度。

这个方向非常对。你看这个最基础的买卖和爆仓逻辑，咱们下一步是先引入订单簿(**Orderbook**)进行多空撮合，还是先加入资产池(**Vault**)来做像 **GMX** 那样的 LP 交易对手模型？

---

*User prompt:* 不，我的意思是，我们要去证明我们的框架可以用实验去观察，我是为了这个

*Response:* 明白了！我刚才有些心急，直接跳到具体的业务逻辑里去了。你的真正意图是“验证‘实验框架’本身的可观察性”。

换句话说，在往里面塞任何复杂的金融逻辑之前，我们得先证明：这个 **Python** 框架能够像一间合格的实验室一样，把市场运行的轨迹、数据的演变、以及多市场并行的状态，清晰、直观、能量化地展示出来。如果连“观察”都做不到，后面做任何实验都是两眼一抹黑。

为了证明这个框架具备“可观察性”，我们需要升级的不是交易逻辑，而是数据的收集与可视化能力。一个合格的实验观察框架，至少要能让我们看到两样东西：

1. 时间序列的连续追踪(看清一个市场随时间变化的曲线)。
2. 多市场的横向对比(证明框架支持并行观察)。

我们可以用 Python 最经典的 `matplotlib` 库，把这个纯代码的实验框架变成一个可视化仪表盘。

## 升级后的“可观察性”实验框架

请确保你的本地环境安装了绘图库：`pip install matplotlib`。

运行下面的代码，它会模拟“市场 A(高波动)”和“市场 B(低波动)”同时运行，并实时把市场的价格曲线和用户的资产价值曲线绘制出来。这就是我们用实验去“观察”的证据：

```
import random
import time
import matplotlib.pyplot as plt

class ExperimentalMarket:
```

```

"""可观察的实验市场：只保留核心的价格变动和资产记录"""
def __init__(self, name, init_price, volatility):
    self.name = name
    self.price = init_price
    self.volatility = volatility # 波动率，用来区分不同市场的特征

    # 实验观察记录本（时间序列数据）
    self.history_time = []
    self.history_price = []
    self.history_account_value = []

    # 模拟一个简单的实验账户：在初始价格时全仓买入
    self.user_shares = 100.0
    self.user_cash = 0.0

def step(self, t):
    """实验推进一步（模拟时间流逝）"""
    # 模拟随机游走价格
    change_pct = random.uniform(-self.volatility, self.volatility)
    self.price = self.price * (1 + change_pct)

    # 计算当前实验账户的总价值
    current_value = (self.user_shares * self.price) + self.user_cash

    # 记录数据（供观察使用）
    self.history_time.append(t)
    self.history_price.append(self.price)
    self.history_account_value.append(current_value)

# ===== 实验启动与观察验证 =====
def run_observation_experiment():
    print("🚀 正在启动‘可观察性’论证实验...")
    random.seed(42) # 固定随机种子，确保实验可重复

    # 1. 初始化两个截然不同的实验市场（证明可以同时观察多个市场）
    market_btc = ExperimentalMarket(name="BTC(高波动市场)", init_price=100.0,
    volatility=0.05)
    market_eth = ExperimentalMarket(name="ETH(低波动市场)", init_price=100.0,
    volatility=0.02)

    # 2. 让实验运行 50 个步长（时间周期）
    for t in range(1, 51):
        market_btc.step(t)
        market_eth.step(t)

    print("📊 实验数据收集完毕，正在生成观察图表...")

    # 3. 绘制观察图表
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)

    # 观察图表 1：价格走势对比
    ax1.plot(market_btc.history_time, market_btc.history_price,

```

```

label=market_btc.name, color='orange', lw=2)
    ax1.plot(market_eth.history_time, market_eth.history_price,
label=market_eth.name, color='blue', lw=2)
    ax1.set_title("观察指标 A: 多市场价格归一化走势对比", fontsize=12)
    ax1.set_ylabel("价格 (USDT)")
    ax1.legend()
    ax1.grid(True, linestyle='--')

    # 观察图表 2: 账户净值追踪
    ax2.plot(market_btc.history_time, market_btc.history_account_value,
label="BTC 实验账户净值", color='darkorange', linestyle='--')
    ax2.plot(market_eth.history_time, market_eth.history_account_value,
label="ETH 实验账户净值", color='darkblue', linestyle='--')
    ax2.set_title("观察指标 B: 实验账户净值动态追踪", fontsize=12)
    ax2.set_xlabel("时间步长 (Steps)")
    ax2.set_ylabel("账户总价值")
    ax2.legend()
    ax2.grid(True, linestyle='--')

plt.tight_layout()
plt.show()
print("✅ 观察图表已成功渲染。实验证明：该框架具备完整的可观察性。")

if __name__ == "__main__":
    run_observation_experiment()

```

## 这段实验证明了什么？

当我们把这段代码跑起来，弹出一个包含两条曲线的图表时，我们就向团队（或者向我们自己）成功证明了以下三点：

- 数据可追溯：我们在内存中成功建立了一个“数据监视器”（history\_price 和 history\_account\_value），市场发生的一切都有迹可循。
- 多市场横向对比：图表中同时出现了 BTC 和 ETH 两个市场，它们互相独立、互不干扰，但可以在同一个时间轴下被观察。
- 逻辑变动的可视化反响：我们可以一眼看出高波动市场带来的资产起伏，明显大于低波动市场。这意味着未来我们只要修改任意一行业务代码（比如调整手续费、调整杠杆、引入爆仓），其产生的连锁反应都能立刻在图表上被直观地捕获到。

这正是“科学实验”该有的样子。有了这个底座，我们后面往里面加任何合约机制，心里都有底了。

你看看目前这个通过图表和时间序列来观察实验的方式，符合你对“可观察性”的预期吗？或者你希望实验还能暴露出哪些更深层的数据指针（比如吞吐量、延迟，或是特定极端情况下的瞬时日志）？

---